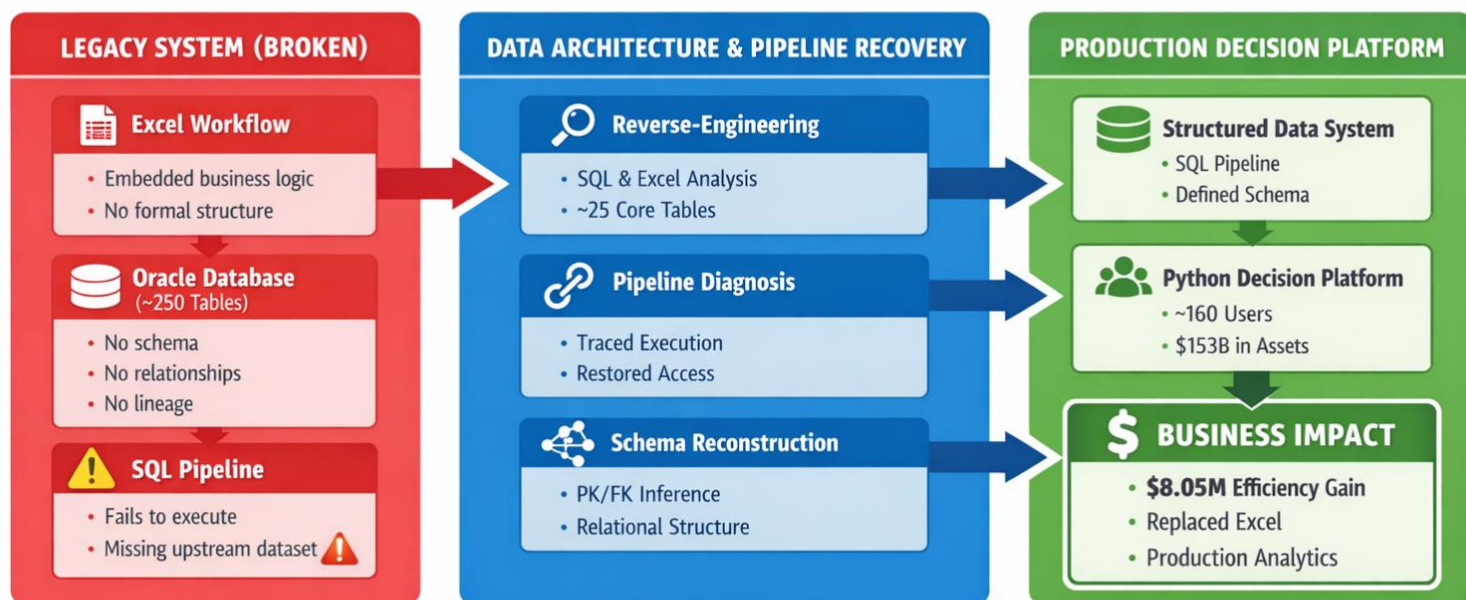


Wellington Management: Data Architecture & Pipeline Recovery



At Wellington Management, I worked on a team migrating a legacy Excel-driven investment workflow into a production Python decision platform used by insurance portfolio managers. The system supported decision-making across approximately \$153B in assets and was relied upon by ~160 investment professionals globally. However, the underlying data environment was highly fragmented. The Oracle database contained roughly 250 tables with no schema, no documented relationships, and no clear data lineage, while the SQL pipeline—originally generated from spreadsheet logic—failed to execute. Before the platform could be rebuilt, the data architecture itself had to be reconstructed.

To address this, I reverse-engineered the system by analyzing both the SQL queries and the embedded business logic in Excel. I identified and prioritized approximately 25 core tables likely to contain the key data dependencies and used targeted SQL inspection queries to examine table structures, infer primary and foreign key relationships, and trace how data flowed through the pipeline. By mapping the database structure against the spreadsheet calculations, I reconstructed the relational schema required to support the system's analytics layer, effectively creating the first coherent model of the underlying data architecture.

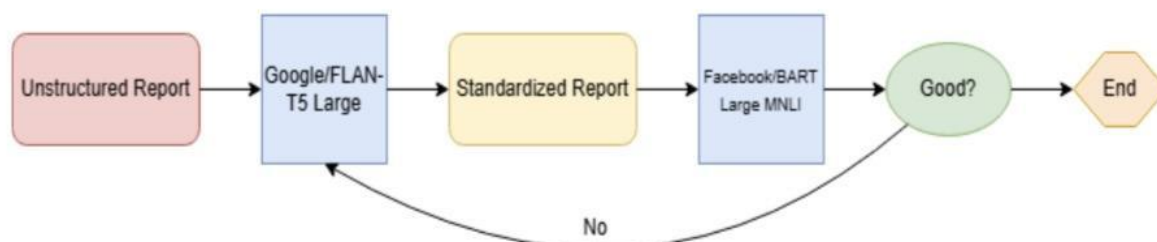
While tracing execution paths, I identified the root cause of the pipeline failure: a missing upstream dataset referenced by the SQL logic that the team did not have access to. Because this dataset was required for downstream transformations, its absence caused the entire pipeline to fail. After diagnosing the issue, I worked with the team to restore access to the missing dependency, enabling the pipeline to execute successfully with minimal modification.

With the pipeline functioning and the schema reconstructed, the team was able to proceed with building the Python-based decision platform. I also contributed to the implementation of an analytics override framework that allowed investment professionals to adjust model inputs while enforcing validation and maintaining auditability. This system was built using Python,

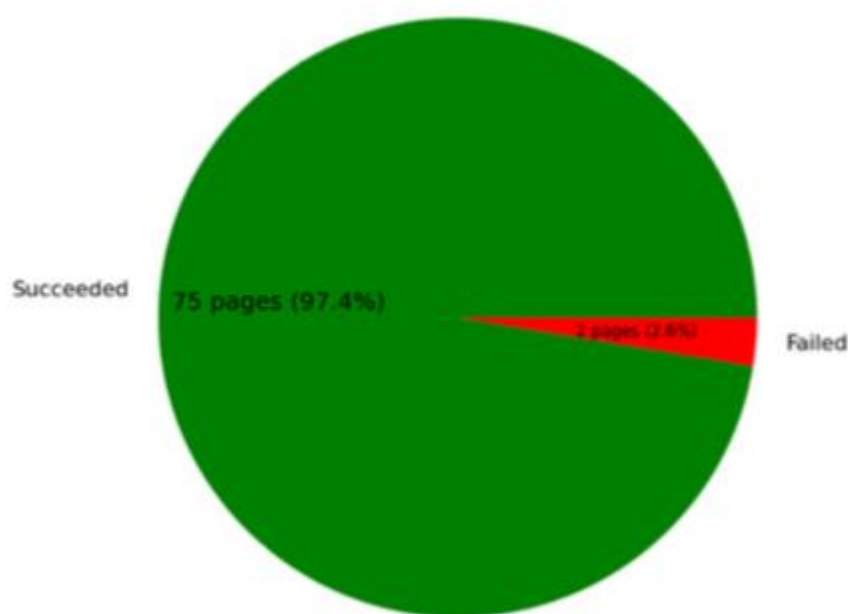
Pandas, Oracle SQL, and ipywidgets, enabling interactive yet controlled decision-making within the platform. The final system replaced a fragile spreadsheet-based workflow with a structured data pipeline and application layer, supporting ~\$153B in assets and delivering approximately \$8M in annual operational efficiency gains.

MassDEP: LLM-Based Document Intelligence System

Model Pipeline



Readability Status of Pages Processed by Model



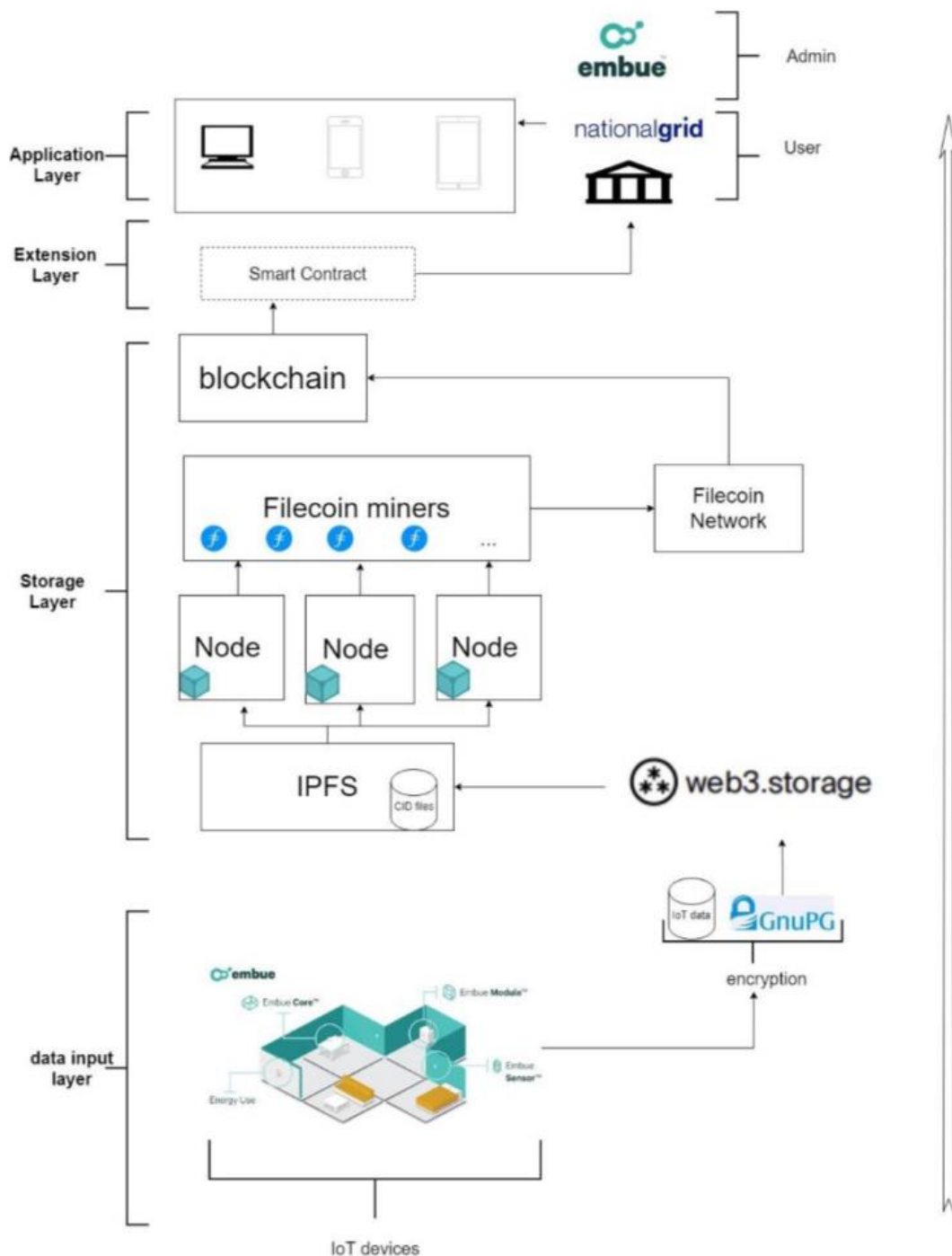
At MassDEP, I worked on a team where I architected and designed a data system to transform unstructured environmental reporting workflows into structured, validated datasets for analysis. Environmental construction reports contained critical data such as PFAS metrics and shipment quantities embedded in inconsistent free-text formats, forcing stakeholders to rely on manual, error-prone extraction. I translated this fragmented, document-based process into a system capable of producing consistent, queryable outputs, shifting the workflow from ad hoc document review to structured data analysis.

I designed and built a multi-stage pipeline combining retrieval, structured extraction, and validation to ensure outputs were both accurate and auditable. I engineered semantic retrieval using vector embeddings to ground outputs in source documents and implemented constrained generation to eliminate unsupported results. I developed a hybrid entity extraction framework and designed validation layers—including rule-based checks, contextual disambiguation, and

post-generation verification—to ensure extracted data was reliable and interpretable. This transformed the system from an experimental NLP workflow into a controlled data pipeline with measurable reliability.

I integrated and optimized the system’s outputs into a Power BI analytics layer, designing dashboards that enabled stakeholders to interact directly with processed data rather than raw documents. This included building page-level inspection capabilities, readability tracking, and system performance visualizations, achieving approximately 97.4% document processing coverage. By connecting the backend pipeline to a user-facing analytics interface, I improved transparency and usability, reduced manual reporting effort by an estimated ~40%, and enabled scalable analysis across environmental reports.

Embue: Distributed IoT Data Infrastructure (Blockchain + IPFS)



At Embue, I designed and implemented a distributed data infrastructure for storing and verifying IoT telemetry generated by environmental sensors in apartment buildings. The core challenge was that traditional IoT pipelines rely on centralized storage systems, which introduce single points of failure and provide no guarantees around data integrity or auditability. If compromised, these systems allow data to be altered without detection, creating risk in environments where data authenticity is critical. The objective was to design a system that ensures tamper-resistant storage, secure data sharing, and verifiable data integrity while remaining scalable for real-world data volumes.

To solve this, I designed a layered architecture that separates the system into four components: data input, storage, extension, and application layers. At the input layer, IoT telemetry is collected from distributed devices and encrypted using GnuPG to ensure confidentiality before entering any public infrastructure. This encryption step was critical because decentralized storage systems such as IPFS are inherently public. The encrypted data is then passed to the storage layer, where it is uploaded via the Web3.Storage API and converted into a content identifier (CID), enabling content-addressed storage across IPFS nodes. Rather than storing raw data directly on-chain—which is not scalable—I designed the system to use IPFS for distributed storage and Filecoin for verification. Filecoin miners store and seal the data while recording storage commitments on the blockchain, ensuring that the data remains verifiable over time.

A key architectural decision was to separate storage from verification. Through analysis of Filecoin’s performance characteristics, I determined that storing large volumes of data directly on-chain was infeasible due to extreme latency and scalability limitations. Instead, the system uses blockchain as a verification layer while keeping bulk data off-chain, enabling both scalability and integrity. The extension layer coordinates interactions between users and the storage system through smart contracts, which enforce access control. When a user requests access to specific data, the system routes the request through an admin approval process. If approved, the smart contract triggers retrieval of the corresponding CID-referenced data from IPFS, which is then decrypted locally by the authorized user. If access is denied, retrieval does not occur, ensuring controlled and auditable data access.

From an infrastructure standpoint, I implemented and configured the environment required to support this architecture, including IPFS nodes, integration with Web3.Storage, and a Filecoin (Lotus) lite client to interact with the network without requiring enterprise-scale hardware. I also configured wallet infrastructure for storage operations and conducted end-to-end testing across environments, validating that encrypted data could be uploaded, distributed across nodes, retrieved via CID, and decrypted only by authorized users. This confirmed both the integrity guarantees provided by content-addressing and the confidentiality enforced through encryption.

The resulting system demonstrates a scalable, tamper-resistant data pipeline that separates encryption, storage, verification, and access control into distinct layers. By combining decentralized storage with blockchain-backed verification and programmatic access control, the architecture provides a generalizable framework for building secure, auditable data systems in domains where data integrity and trust are critical, including IoT platforms, financial systems, and regulatory environments.

BoardGameGeek: Data Engineering & Full-Stack Pipeline

In the BoardGameGeek project, I built a data pipeline and supporting application components to collect, process, and integrate board game data from multiple sources into a structured system. The primary challenge was integrating heterogeneous datasets—marketplace prices, ratings, and metadata—while dealing with rate-limited external APIs and inconsistent data formats.

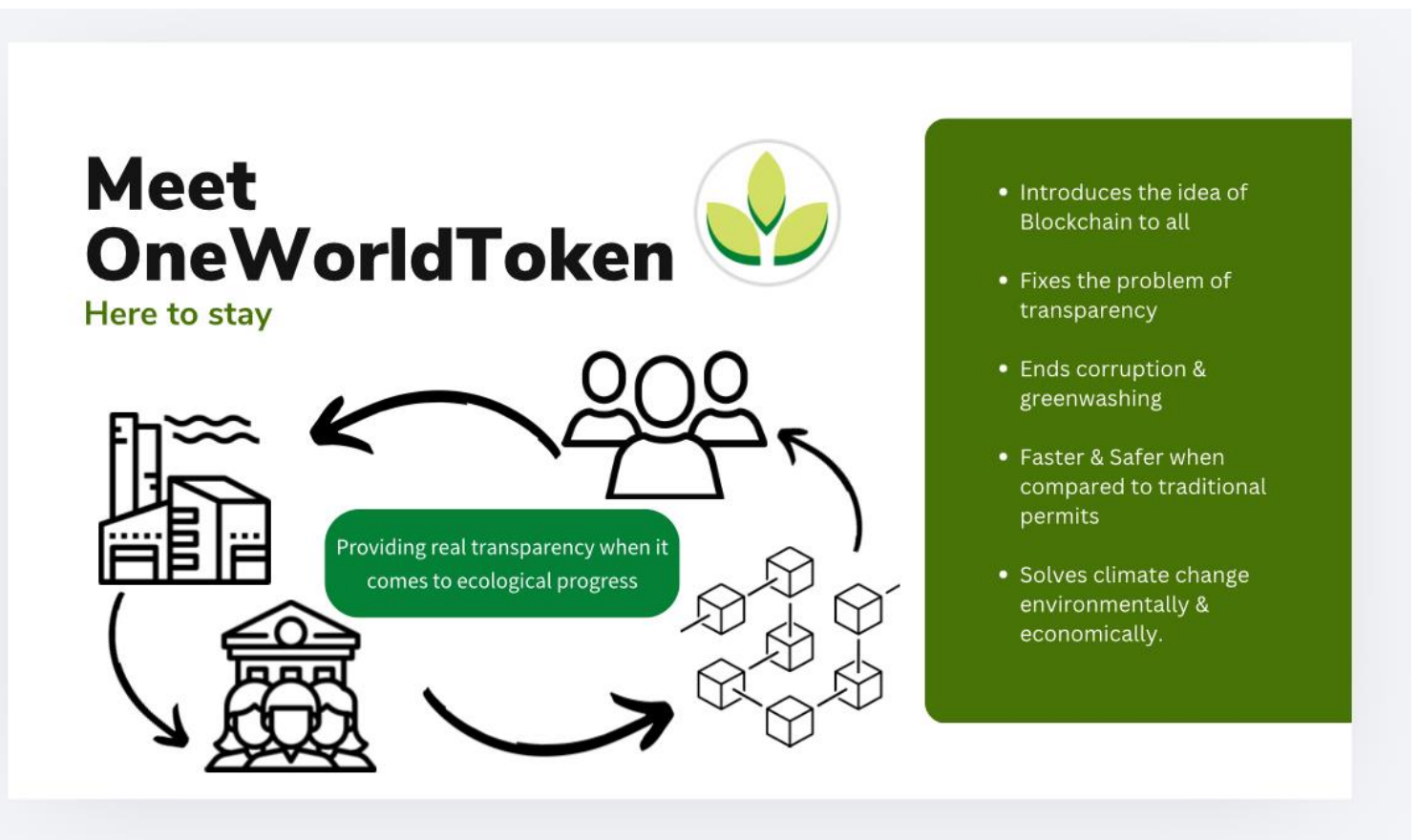
I developed a three-stage asynchronous data pipeline in Python using aiohttp to enable concurrent API requests and improve ingestion throughput. The ingestion layer included rate-limit handling, retry logic, and progress tracking to ensure reliable data collection across approximately 150 board games. The pipeline then entered a data integration phase, where pricing data was merged with metadata and ratings datasets. This required resolving missing values, standardizing schema structures, aligning data types, and validating data integrity before merging records.

In the final stage, I implemented normalization and validation processes to ensure the dataset could be reliably stored and queried. This included reindexing ranking data to maintain consistency and restructuring the schema to support more complex relationships. I introduced a recursive data model to represent relationships between board games, such as reimplementations and expansions, replacing a simplistic boolean approach with a more accurate relational structure.

In addition to the data pipeline, I implemented a serverless AWS Lambda function to handle account balance updates and developed a React-based checkout workflow that enabled users to manage carts and complete transactions. This connected the data pipeline to backend services and frontend interactions, resulting in a full-stack system that integrates data ingestion, processing, and application functionality.

OneWorld: Tokenized Carbon Market Infrastructure

OneWorld is a blockchain-based carbon market infrastructure designed to address structural failures in traditional cap-and-trade systems, including lack of transparency, susceptibility to manipulation, and weak enforcement of emissions constraints. The system digitizes emissions permits into programmable financial assets, where each token (OneWorldToken, OWT) represents one metric ton of CO₂. Unlike traditional implementations, where permit allocation and trading occur through opaque, manually governed processes, OneWorld embeds the full lifecycle of emissions permits—issuance, trading, and retirement—directly into a transparent, on-chain system. This system was developed as part of a competitive blockchain hackathon, where it placed Top 3 out of 100+ teams.



At the core of the design is a programmatically enforced emissions reduction schedule. Tokens are issued on a recurring basis with a deterministic supply decay of approximately 2.11% per quarter, derived from historical emissions data and calibrated to align with global climate targets such as the Paris Climate Agreement. This removes

reliance on external policy enforcement by encoding environmental constraints directly into the system's economic infrastructure. A key design decision was to make tokens non-fungible across time periods, preventing firms from hoarding permits during high-supply periods and using them later to bypass emissions constraints. This directly addresses inefficiencies in traditional systems, where predictable supply reductions create opportunities for speculative behavior and distort market dynamics.



The system preserves the economic structure of cap-and-trade markets while improving their enforcement and transparency. Firms are required to hold tokens corresponding to their emissions and can trade permits in an open market. More efficient firms that reduce emissions below their allocation can sell surplus tokens, while less efficient firms must purchase additional permits. To strengthen incentives, OneWorld incorporates a tiered pricing model in which the marginal cost of emissions increases as firms consume more permits, aligning economic behavior with environmental constraints while maintaining market efficiency.

From a technical perspective, the system leverages blockchain infrastructure for verification, auditability, and enforcement, rather than heavy computation. Built on Algorand's carbon-neutral, pure proof-of-stake network, OneWorld ensures that the infrastructure itself does not introduce additional environmental costs. All token activity is recorded on-chain, creating a transparent and immutable ledger of emissions behavior that can be audited in real time by regulators, participants, and external stakeholders. This directly addresses issues such as double-counting, fraudulent reporting, and lack of visibility that have historically undermined trust in carbon markets. The architecture is designed to integrate with decentralized oracle networks, enabling ingestion of real-world emissions data and reducing reliance on manual reporting.

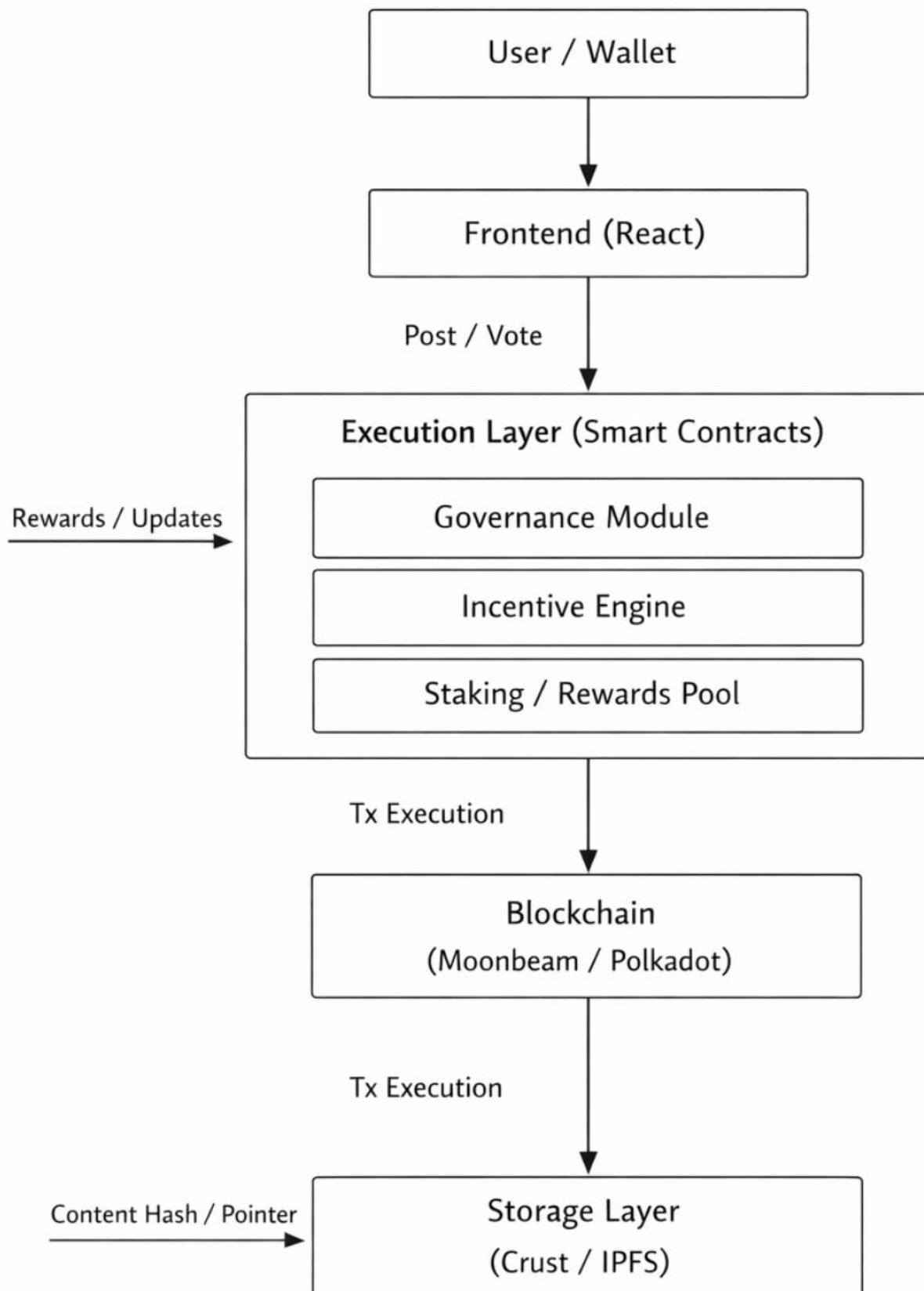
The system also incorporates constraints to mitigate market manipulation and game-theoretic vulnerabilities, including restrictions on token usage across time periods and structural limits on transaction behavior. By combining economic modeling with programmable infrastructure, OneWorld transforms emissions markets from loosely enforced policy frameworks into mathematically constrained systems where incentives, behavior, and outcomes are aligned by design.

The primary tradeoff of the system is its dependence on regulatory adoption, as emissions markets are inherently tied to government policy. Additional challenges include potential liquidity constraints in early stages and the need for reliable external data sources to fully automate enforcement. To address these constraints, a phased deployment strategy is proposed, beginning with smaller-scale pilot implementations to validate system behavior before scaling to broader regulatory environments.

Next steps for development include building a simulation engine to model pricing dynamics and participant behavior, developing a prototype token marketplace, and integrating real-world emissions data through oracle systems. More broadly, OneWorld represents a shift from policy-driven enforcement to infrastructure-driven enforcement, where environmental constraints are embedded directly into the economic and technical systems governing emissions markets.

Oasis: Decentralized Social Platform with Tokenized Governance

Oasis is a decentralized social media platform designed to give users ownership and control over content through a DAO-based governance and incentive system. Developed in a 48-hour hackathon environment, the system was built under tight time constraints and placed 1st out of 100+ teams. The platform enables users to post content anonymously while participating in a community-driven moderation and reward system, addressing issues such as centralized control, content censorship, and misaligned platform incentives in traditional social media.



The system architecture combines smart contracts, decentralized storage, and token-based incentives to create a self-governing ecosystem. Users submit posts and comments that are evaluated through a real-time, point-based voting system, where every participant contributes to content validation through DAO governance. Content quality is determined by net community voting, and users are rewarded with platform tokens (DEV) based on engagement and reception, introducing a direct economic incentive for meaningful contributions.

A key design component is the staking and reward distribution system, where buy-in tokens from user participation are aggregated into a shared reward pool and distributed periodically based on DAO voting outcomes. This creates a closed-loop incentive structure where user activity directly influences both governance and economic outcomes. Data related to posts, votes, and user interactions is stored using decentralized storage solutions (Crust), ensuring persistence while preserving user anonymity.

From a system design perspective, Oasis balances decentralization with usability by integrating a simplified front-end interface with blockchain-based back-end infrastructure. The platform also incorporates mechanisms to preserve anonymity while maintaining accountability through reputation and voting behavior. Additional planned features include time-bound content visibility (24-hour post lifecycle), LLM-based content assistance, and automated moderation systems to flag potentially harmful content for community review.

The primary tradeoff of the system is the complexity of coordinating governance, incentives, and user experience within a decentralized environment, particularly under strict development time constraints. While the prototype demonstrates the feasibility of the architecture, further work would be required to harden smart contracts, scale the infrastructure, and validate user behavior in a real-world setting. Oasis ultimately demonstrates the ability to translate abstract system requirements into a functional, end-to-end product under significant time pressure while preserving core design integrity.